Weighted Round Robin (WRR) Implementation Project

---

1. Introduction

This report presents the modeling, simulation, and performance analysis of Weighted Round Robin (WRR) and Class-Based Queuing (CBQ) scheduling algorithms using OMNeT++. The purpose of the study is to demonstrate the WRR algorithm in a given small network.

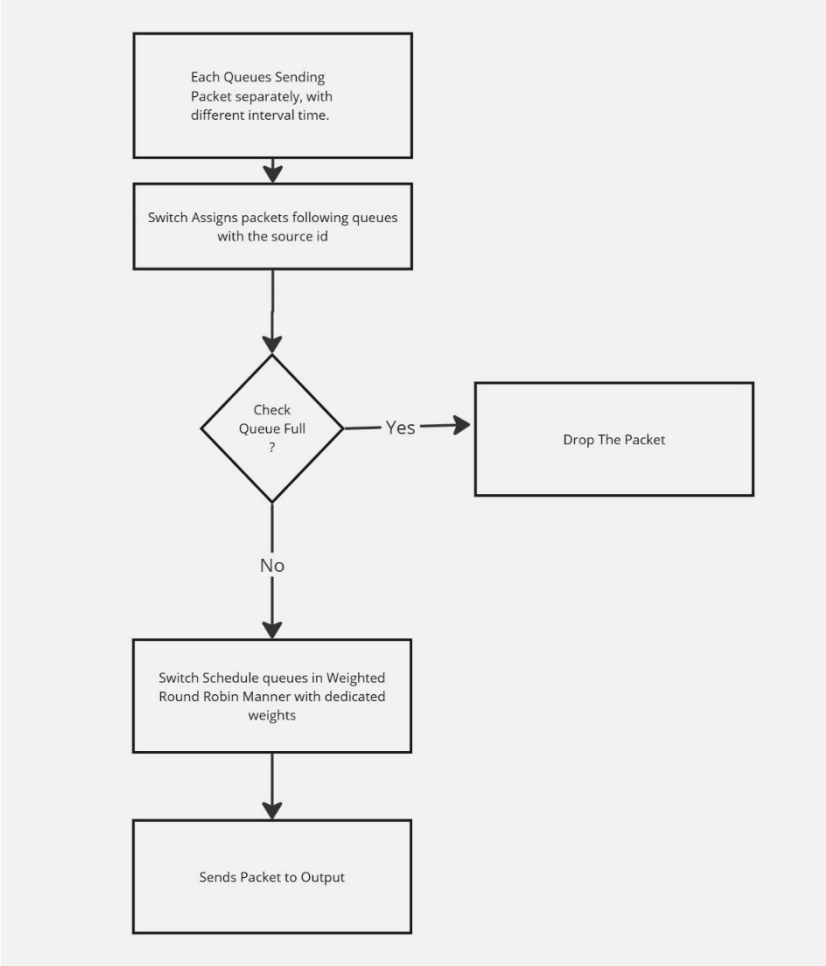
- Client Class
- Switch Class
- Server Class

The Purpose of Client Class is to send packets in a different interval time to Switch. With exponential () function, we elaborate on the purpose of possibility distribution.

Right After the packets received by Switch, first create queues and assigned the corresponding weights. Switch gets the information of the Gate Id to classify the packets in respective queues. After Classifying packets with respecting queues, The Scheduler Function begins. The purpose of the scheduler function is to implement Weighted Round Robin in which in each cycle, with respected weights, higher weights corresponding higher bandwidth in the switch-server bandwidth, packets are sent. For example, higher bandwidth queues are visited multiple times in a single cycle. For each time, the switch must check whether the queue of the client is full or not, if the queue is full, the upcoming packet will be drop otherwise Weighted Round Robin algorithm continues. To Implement the structure of Switch, I created 4 functions which are initialize (), handleMessage (cMessage *msg), processQueues (), and finish () functions. In Initialize function, I initialized the variables of the Switch Class, and assigned the values of .ini , and created 4 queues respectively. In handleMessage function, the purpose is to get the packet, assigned to respective queue, as classification. After classifying the packet, the processQueues function starts, in which Weighted Round Robin is implemented. Finishing function allows to clear the memory space by deleting the values of each queue.

Server Class only receives the packets and deletes them.

2. Flowchart of Implementation



3. Test Parameters

Parameter	Value	Description
Number of Clients	**4**	**Total number of input queues**
Transmission Rates between Switch and Server	**100 Mbps**	**Rate tested for performance**
Queue Weights	**40 %,30%,20%,10%,**	**Weighted round-robin distribution weights**
Maximum Capacity Queue	**10**	**Maximum capacity of the queues**

4. Performance Analysis

4.1 Throughput for Different Clients

The Throughput for Different Clients represents the difference between the weights of each client, because the packet number they sent in each cycle is related to the weights respectively.

5. Conclusion

This study provides insights into how WRR scheduling algorithms impact network performance in a client-server topology. The key findings from the simulations are that The WRR algorithm can improve throughput for higher-weight clients while respecting the fairness of lower-weight clients due to round robin manners, Queue Management prevents congestion by efficiently managing queues, and The Higher Transmission rates between the switch and the server reduce the queue sizes and increase throughput.